

Real-Time Crime Analytics & Intelligent Alert System

Final Project Report

Group Members Sameed Ahmed Siddiqui; Sameed Ahmed Siddiqui
- [22i-8223] Junaid Zeb; Junaid Zeb
- [22i-8794]

May 7, 2026

Abstract

This report details the design and implementation of a Real-Time Crime Analytics and Intelligent Alert System utilizing public safety datasets from the City of Chicago. Employing a Lambda Architecture, the system simultaneously processes historical data in batches using Apache Spark and streaming data in real-time using Apache Kafka and Apache Storm. It features K-Means geospatial clustering for crime hotspot detection, complex cross-dataset joins, and dynamic anomaly detection for district-level crime spikes. All components were containerized using Docker, allowing seamless cross-platform execution.

Contents

1	Introduction	3
2	System Design & Architecture	3
2.1	Batch Layer (Apache Spark)	3
2.2	Speed Layer (Apache Kafka & Apache Storm)	3
2.3	Serving Layer (PostgreSQL & MongoDB)	4
2.4	Infrastructure & Folder Structure	4
3	Schema Descriptions & Preprocessing	5
3.1	Dataset Schemas	5
3.2	Preprocessing Pipeline	5
4	Analytics Methodology & Batch Results	6
4.1	Crime Trend Analysis	6
4.2	Arrest Rate Analysis	6
4.3	Violence, Gunshot & Sex Offender Proximity Analysis	6
4.4	Cross-Dataset Correlations	6
5	Machine Learning Approach: Geospatial Hotspots	7
5.1	K-Means Clustering	7

6	Real-Time Streaming & Alerts	8
6.1	Stream Simulation and Topology	8
6.2	Anomaly Detection Logic	8
7	Challenges Faced & Technical Solutions	9
7.1	Overcoming Apache Storm & Streamparse Limitations	9
7.2	Docker Networking for Distributed Bolts	9
7.3	Windows Migration	9
8	Integration of Generative AI	10
9	Conclusion	10

1 Introduction

Urban crime analysis is a fundamental pillar of public safety. However, traditional batch-processing pipelines are inadequate for time-sensitive decision-making. Modern smart cities require real-time analytics to detect anomalies, identify hotspots, and dispatch alerts immediately as incidents occur.

In this project, we built a comprehensive Big Data engineering pipeline that leverages the Lambda Architecture to process Chicago's public safety datasets. Our system integrates both historical analytics and real-time streaming, enabling robust law enforcement analytics.

2 System Design & Architecture

The core of our platform relies on the **Lambda Architecture**, guaranteeing fault tolerance, scalability, and support for both batch and real-time analytics.

2.1 Batch Layer (Apache Spark)

The batch layer is responsible for processing historical datasets, enforcing explicit schemas, and calculating overarching trends. We utilized **Apache Spark** (PySpark) to ingest multi-schema heterogeneous CSV datasets from disk. Key operations included:

- Explicit schema enforcement using **StructType**.
- Cleansing missing values and casting data types.
- Computing historical trends, arrest rates, and cross-dataset correlations.
- Applying Unsupervised Machine Learning (K-Means) for geospatial hotspot detection.

2.2 Speed Layer (Apache Kafka & Apache Storm)

The speed layer handles real-time data ingestion and low-latency processing.

- **Kafka Producer:** Simulates live incident feeds by streaming crime records row-by-row into the `crime_events` topic.
- **Apache Storm Topology:** A multi-bolt architecture processes the stream:
 1. **KafkaSpout:** Consumes messages from the Kafka topic.
 2. **ParseBolt:** Deserializes JSON and validates required fields.
 3. **DistrictBolt:** Routes events by police district.
 4. **WindowBolt:** Implements a sliding time window to count events per district.
 5. **AnomalyBolt:** Compares windowed counts against a baseline threshold to flag anomalies.
 6. **AlertBolt:** Persists triggered alerts to the Serving Layer.

2.3 Serving Layer (PostgreSQL & MongoDB)

Processed results from both the batch and speed layers converge in the serving layer:

- **PostgreSQL:** Stores structured, relational outputs such as crime trends, hotspots, cross-dataset correlations, and alert history.
- **MongoDB:** Acts as a document store for semi-structured data, capturing raw event logs and full JSON payloads of real-time alerts.

2.4 Infrastructure & Folder Structure

The entire architecture is containerized using Docker and Docker Compose. We developed a custom PowerShell script (`run.ps1`) to orchestrate the infrastructure on Windows, replacing the traditional Makefile approach.

Directory Structure:

```
BigData-Project/
├── config/ ... Configuration files (config.yaml)
├── dashboard/ ... Streamlit dashboard (app.py)
├── data/ ... Chicago Crime CSV datasets
├── db/ ... Database initialization scripts
├── docker/ ... docker-compose.yml and custom Dockerfiles
├── kafka/ ... Kafka Producer scripts
├── spark/ ... PySpark batch analytics scripts
├── storm/ ... Storm topologies, spouts, and bolts
├── run.ps1 ... Windows execution orchestrator
└── README.md ... Project documentation
```

3 Schema Descriptions & Preprocessing

One of the primary challenges involved handling dirty, heterogeneous data. Schema inference (`inferSchema=True`) was strictly avoided in favor of explicitly defined `StructType` schemas.

3.1 Dataset Schemas

- **Crime Data:** Defined with fields including CASE NUMBER, DATE, BLOCK, PRIMARY TYPE, DISTRICT, ARREST, LATITUDE, and LONGITUDE.
- **Arrests Data:** Mapped to capture CB NO, CHARGE STATUTE, CHARGE DESCRIPTION, and CHARGE TYPE.
- **Violence & Sex Offenders:** Joined on location and district data to analyze localized risks and demographic factors.

3.2 Preprocessing Pipeline

Our `load_and_clean_data` function in PySpark performs several crucial cleaning steps:

```
1 from pyspark.sql.types import StructType, StructField, StringType,
   DoubleType, BooleanType
2
3 crime_schema = StructType([
4     StructField("CASE NUMBER", StringType(), True),
5     StructField("DATE", StringType(), True),
6     StructField("BLOCK", StringType(), True),
7     StructField("PRIMARY TYPE", StringType(), True),
8     StructField("DISTRICT", StringType(), True),
9     StructField("ARREST", BooleanType(), True),
10    StructField("LATITUDE", DoubleType(), True),
11    StructField("LONGITUDE", DoubleType(), True)
12 ])
13
14 df_crime = spark.read.csv(
15     "/data/crime.csv",
16     schema=crime_schema,
17     header=True
18 )
```

Listing 1: PySpark Schema Enforcement Example

Null coordinate filtering, timestamp parsing, and column renaming were executed to prepare the DataFrames for joins and machine learning.

4 Analytics Methodology & Batch Results

Using PySpark, we executed five major batch analytics tasks, resulting in structured relational tables stored in PostgreSQL.

4.1 Crime Trend Analysis

We aggregated crime counts based on temporal dimensions: Year, Month, Day of the Week, and Hour of the Day. This allows law enforcement to deploy patrols dynamically during peak hours (e.g., Friday nights at 23:00).

4.2 Arrest Rate Analysis

By executing a `left_outer` join between the Crime and Arrests datasets on `CASE NUMBER`, we computed the arrest rate (total arrests / total crimes) grouped by primary crime type, police district, and race.

Rank	Primary Crime Type	Arrest Rate
1	NARCOTICS	0.985
2	PROSTITUTION	0.970
3	WEAPONS VIOLATION	0.765
4	CRIMINAL TRESPASS	0.720
5	PUBLIC PEACE VIOLATION	0.680

Table 1: Top 5 Crime Types by Arrest Rate

4.3 Violence, Gunshot & Sex Offender Proximity Analysis

The violence dataset was grouped by month and district to separate total homicides from non-fatal shootings, including a calculation of the proportion of incidents involving confirmed gunshot injuries (`GUNSHOT INJURY I = 'Y'`). Furthermore, by joining the Sex Offenders dataset with the Police Stations on `DISTRICT`, we identified districts with the highest density of registered offenders, explicitly flagging records where `VICTIM MINOR = 'Y'`.

4.4 Cross-Dataset Correlations

We discovered statistical relationships between disparate datasets using PySpark's `corr` function:

- **Violence vs Arrest Rate:** Revealed whether high-violence districts experience proportionally lower or higher arrest rates.
- **Offender Density vs Crime Count:** Established a baseline understanding of whether the density of registered sex offenders correlates with overall district crime volume.

5 Machine Learning Approach: Geospatial Hotspots

To facilitate proactive policing, we implemented Unsupervised Machine Learning using PySpark's MLlib.

5.1 K-Means Clustering

We applied the K-Means algorithm to partition crime events into localized geographic clusters.

1. **Feature Engineering:** Using `VectorAssembler`, we combined `LATITUDE` and `LONGITUDE` into a unified feature vector.
2. **Model Training:** We initialized the K-Means algorithm with $k = 10$ (representing 10 distinct high-risk zones across Chicago) and a fixed seed for reproducibility.
3. **Centroid Extraction:** The model output provided the centroids of the 10 most prominent crime hotspots.

```
1 assembler = VectorAssembler(inputCols=["LATITUDE", "LONGITUDE"],
2   outputCol="features")
3
4 dataset = assembler.transform(coords)
5
6 kmeans = KMeans(k=10, seed=1)
7 model = kmeans.fit(dataset)
8 predictions = model.transform(dataset)
9
10 hotspots = predictions.selectExpr("prediction as cluster_id", "LATITUDE
11   as latitude", "LONGITUDE as longitude")
12 hotspots.write.jdbc(url=db_url, table="hotspots", mode="overwrite",
13   properties=db_properties)
```

Listing 2: K-Means Hotspot Detection

The output from this ML model is directly exposed via the Streamlit dashboard, overlaying the coordinates on an interactive map.

6 Real-Time Streaming & Alerts

The speed layer provides immediate situational awareness by identifying short-term spikes in crime activity.

6.1 Stream Simulation and Topology

The `kafka_producer.py` script reads the historical CSV row-by-row and emits JSON messages to the `crime_events` Kafka topic at a rate of 100 message/second.

Our Storm Topology processes these events via `streamparse`:

```
1 class CrimeTopology(Topology):
2     kafka_spout = CrimeKafkaSpout.spec()
3     parse_bolt = ParseBolt.spec(inputs={kafka_spout: Grouping.SHUFFLE})
4     district_bolt = DistrictBolt.spec(inputs={parse_bolt: Grouping.
5     SHUFFLE})
6     window_bolt = WindowBolt.spec(inputs={district_bolt: Grouping.
7     fields('district')})
8     anomaly_bolt = AnomalyBolt.spec(inputs={window_bolt: Grouping.
9     fields('district')})
10    alert_bolt = AlertBolt.spec(inputs={anomaly_bolt: Grouping.SHUFFLE
11    })
```

Listing 3: Storm Topology Architecture

6.2 Anomaly Detection Logic

The `WindowBolt` maintains a sliding time window. Every time the window slides, it emits the event count per district. The `AnomalyBolt` evaluates this count against a predefined threshold (e.g., 5 events per 5 minutes in a single district). If the threshold is breached, an anomaly tuple is emitted to the `AlertBolt`, which subsequently writes a permanent record to MongoDB and PostgreSQL.

7 Challenges Faced & Technical Solutions

Deploying a full-scale Big Data environment locally introduced significant architectural challenges, specifically regarding Docker networking, Windows execution, and Python dependency management.

7.1 Overcoming Apache Storm & Streamparse Limitations

The `streamparse` library natively expects to deploy code by SSH-ing into a cluster of remote servers and creating Python virtual environments. In our containerized environment, the Storm `nimbus` and `supervisor` containers did not have SSH enabled, causing the topology submission to crash with `FileNotFoundError` and SSH timeout errors.

Solution: We bypassed the SSH and virtualenv requirements by architecting a custom Dockerfile (`Dockerfile.storm`) for the Storm nodes. We installed `pip3` and directly injected the required dependencies (like `kafka-python` and `psycopg2`) onto the Java Storm nodes. We then updated `storm/config.json` to include `"use_ssh_for_nimbus": false` and `"use_virtualenv": false`.

7.2 Docker Networking for Distributed Bolts

When the Storm `supervisor` node executed the Python spouts and bolts, the Python code attempted to connect to Kafka and MongoDB using `localhost`. Because the Python processes were running *inside* the supervisor container, `localhost` referred to the supervisor itself, resulting in connection refusals.

Solution: We injected the `IN_DOCKER=1` environment variable into the `nimbus` and `supervisor` containers via `docker-compose.yml`, ensuring the Python bolts resolved connections using the Docker DNS (e.g., connecting to `kafka:9092` instead of `localhost:29092`).

7.3 Windows Migration

The original project utilized a `Makefile` containing `sudo` and `bash` commands, which fundamentally broke on Windows machines. **Solution:** We developed `run.ps1`, a comprehensive PowerShell orchestrator that handled all Docker `exec` commands, passed necessary environment variables like `LEIN_ROOT=yes`, and successfully deployed the pipeline on Windows without relying on WSL Make tools.

8 Integration of Generative AI

In accordance with course guidelines, Generative AI tools (specifically Google’s Gemini-powered Antigravity agent) were utilized to assist in the development and debugging phases.

Key Areas of AI Assistance:

- **Cross-Platform Migration:** The AI generated the `run.ps1` PowerShell script, enabling the seamless migration of the Linux-based `Makefile` infrastructure to a native Windows environment.
- **Complex Debugging:** When the `streamparse` library failed during topology submission, the AI analyzed the worker logs, identified the missing SSH/virtualenv architecture inside the Docker containers, and wrote the custom `Dockerfile.storm` to resolve the dependency issues.
- **Streamlit Dashboard Optimization:** The AI diagnosed a `streamlit` version incompatibility regarding the `@st.fragment` decorator and assisted in refining the real-time alert polling logic from MongoDB.
- **Documentation:** The AI assisted in formatting and generating the initial drafts of the $\LaTeX$ reporting structure, ensuring all rubrics and system design components were covered.

All underlying business logic, architectural decisions, dataset joins, PySpark transformations, and K-Means implementations were conceptualized and validated by the group members.

9 Conclusion

The Real-Time Crime Analytics system successfully achieved all project objectives. By unifying batch analysis and real-time streaming within a Lambda Architecture, we transformed disparate, heterogeneous public safety datasets into actionable intelligence. The use of K-Means clustering provided proactive geographical insights, while the sliding-window anomaly detection ensured immediate notification of high-risk activity. The robust containerized deployment guarantees that this pipeline can be reproduced, scaled, and analyzed reliably across varying environments.